

Business Requirements Document (BRD)

Project Name: E-Commerce Portfolio Platform Version: 1.0 Date: [Insert Date] Author: [Your Name/Team]

1. Executive Summary

Project Overview

The **E-Commerce Portfolio Platform** is a **full-stack web application** designed to enable businesses to manage their online storefronts, including product catalogs, user accounts, orders, invoicing, and geographic location management. This platform supports **administrative, staff, and customer roles** with distinct permissions and workflows.

Business Objectives

- Provide a **scalable, modular e-commerce backend** for managing products, orders, and user interactions.
- Enable **role-based access control** (admin, staff, customer) for secure data management.
- Support **real-time inventory tracking** and **order processing** with automated invoicing.
- Facilitate **geographic location management** (states, cities) for shipping and logistics.
- Ensure **scalability and maintainability** through a well-structured codebase and modular architecture.

Expected Outcomes

- A **fully functional e-commerce backend** integrated with a **React-based frontend**.
- **Automated email notifications** for order confirmations, invoices, and password resets.
- **Role-based dashboards** for admins, staff, and customers.
- **Seamless API integration** for future extensions (e.g., payment gateways, CRM systems).

2. Project Scope

In-Scope Features

Feature Area	Description
User Management	Registration, login, password reset, role-based access (admin, staff, customer).
Product Catalog	CRUD operations for products (name, description, price, images, category).
Order Processing	Order creation, status tracking (pending, completed, cancelled), inventory updates.
Invoicing	Invoice generation, payment status tracking, and deletion.
Geographic Management	State and city registration for shipping and logistics.
Shopping Cart	Add/remove items, clear cart, and calculate total price.
Email Notifications	Automated emails for order confirmations, invoices, and password resets.
Admin Dashboard	

Analytics, user management, product inventory, and order tracking. | | **API Integration** | RESTful API for frontend communication (Next.js). |

Out-of-Scope Items

- **Multi-currency support** (single currency assumed).
- **Advanced analytics** (e.g., sales trends, customer behavior).
- **Mobile app development** (web-focused only).
- **Third-party payment gateways** (e.g., Stripe, PayPal) integration.
- **Multi-language support** (English-only).

Key Assumptions

- **MongoDB** will be used for data storage (NoSQL flexibility).
- **JWT (JSON Web Tokens)** will handle authentication.
- **React (Next.js)** will be the primary frontend framework.
- **Express.js** will serve as the backend API.
- **Nodemailer** will handle email sending (SMTP-based).
- **Soft deletes** will be used for data integrity (e.g., users, products, orders).

3. Business Requirements

3.1 Functional Requirements

User Management

| Requirement | Description | |-----|-----|
-----| | **User Registration** | Allow new users to sign up with email, password, and contact details. | | **User Login/Logout** | Secure authentication with JWT tokens. | | **Password Reset** | Email-based password recovery flow. | | **Role-Based Access** | Admins, staff, and customers have distinct permissions. | | **User Profile Management** | Users can update their personal details (e.g., phone, addresses). |

Product Management

| Requirement | Description | |-----|-----|
-----| | **Product CRUD** | Create, read, update, and delete products with categories. | | **Product Search** | Filter products by name, category, or price range. | | **Inventory Tracking** | Real-time stock updates when orders are placed. | | **Product Images** | Support for multiple product images (e.g., Cloudinary integration). |

Order Processing

Requirement	Description
Order Creation	Users can add products to cart and checkout.
Order Status Tracking	Orders can be marked as pending, completed, or cancelled.
Inventory Deduction	Automatically reduce stock when an order is placed.
Order History	Users can view their past orders.

Invoicing

Requirement	Description
Invoice Generation	Automatically generate invoices for completed orders.
Payment Status	Track whether an invoice is paid or unpaid.
Invoice Deletion	Admins can delete unpaid invoices.

Geographic Management

Requirement	Description
State & City Registration	Admins can add states and cities for shipping addresses.
Address Management	Users can associate addresses with their accounts.

Shopping Cart

Requirement	Description
Add/Remove Items	Users can modify cart contents before checkout.
Cart Persistence	Cart persists across sessions for logged-in users.
Total Price Calculation	Automatically calculate subtotal, tax, and shipping.

Email Notifications

Requirement	Description
Order Confirmation	Email sent after order placement.
Invoice Delivery	Email sent with invoice details.
Password Reset	Email sent with reset link.

Admin Dashboard

Requirement	Description
User Management	Admins can view, edit, and delete users.
Order Tracking	Admins can view all orders and their statuses.
Product Inventory	Admins can manage product stock levels.
Analytics Dashboard	Basic sales and user activity metrics.

3.2 Non-Functional Requirements

Category	Requirement
Performance	API responses should load within < 2s under normal load.
Scalability	System should handle 10,000+ concurrent users without degradation.
Security	- JWT tokens with 1-hour expiry. - Passwords stored as hashed BCrypt (256-bit). - CORS and CSRF protection enabled. - Rate limiting to prevent brute-force attacks.
Availability	99.9% uptime (SLA).
Data Integrity	Soft deletes for users.

products, and orders. | | **Backup & Recovery** | Daily **automated backups** of MongoDB data. | | **Compliance** | **GDPR-compliant** data handling (user data deletion requests). |

3.3 User Stories

User Type	Story	Acceptance Criteria
Customer	As a customer, I want to browse products so I can find what I need.	- Product list page loads with search/filter functionality.
Customer	As a customer, I want to add items to my cart so I can purchase them later.	- "Add to Cart" button works; cart persists across sessions.
Customer	As a customer, I want to checkout with my payment details so I can complete my purchase.	- Checkout flow validates payment info and generates an order.
Customer	As a customer, I want to receive an order confirmation email so I know my purchase was successful.	- Email sent with order details after checkout.
Admin	As an admin, I want to manage products so I can update inventory and pricing.	- CRUD operations for products; stock levels update automatically on orders.
Admin	As an admin, I want to view all orders so I can track sales and resolve issues.	- Admin dashboard shows order history with filters.
Admin	As an admin, I want to generate invoices so I can bill customers.	- Invoice generation with payment status tracking.
Staff	As staff, I want to process orders so I can fulfill customer requests.	- Staff can update order status (e.g., "shipped").
Staff	As staff, I want to manage user accounts so I can support customers.	- Staff can view/edit user details (excluding sensitive data).

4. Technical Architecture Overview

4.1 High-Level System Architecture



Users	Products	Orders, Invoices, Cart, etc.
-------	----------	------------------------------

4.2 Technology Stack

Layer	Technology	Purpose
	Frontend	Next.js (React) SPAs with server-side rendering for SEO and performance.
	Backend	Express.js RESTful API for frontend communication.
	Database	MongoDB (Atlas) NoSQL database for flexible schema and scalability.
	Authentication	JWT (JSON Web Tokens) Secure user authentication and role-based access control.
	Email	Nodemailer SMTP-based email sending for notifications.
	Validation	Zod Runtime type checking and validation for API inputs.
	Testing	Jest, Supertest Unit and integration testing for backend APIs.
	DevOps	Docker, GitHub Actions Containerization and CI/CD pipelines.

4.3 Integration Points

- Third-Party Services:
 - Cloudinary (for product image storage).
 - SMTP Provider (e.g., SendGrid, Mailgun) for email delivery.
- Future Extensions:
 - Payment Gateways (Stripe, PayPal).
 - CRM Integration (e.g., HubSpot, Salesforce).
 - Multi-Language Support (i18n libraries).

5. User Personas & Use Cases

5.1 Target Users

Persona	Description	Key Needs
Admin	High-level user with full access to all features.	- Manage users, products, and orders. - Generate reports and analytics.
Staff	Supports customers with limited permissions.	- Process orders and update statuses. - Assist customers via email/chat.
Customer	End-user purchasing products.	- Browse and search products. - Add items to cart and checkout. - View order history and invoices.

5.2 Primary Use Cases

Use Case 1: Customer Purchases a Product

1. **Precondition:** Customer is logged in or can browse as a guest.
2. **Main Flow:**
 - Customer searches/browses products.
 - Customer adds product to cart.
 - Customer proceeds to checkout.
 - Customer enters payment details and submits order.
3. **Alternative Flows:**
 - Customer abandons cart (persists for logged-in users).
 - Customer edits cart before checkout.
4. **Postcondition:** Order is created; email confirmation sent.

Use Case 2: Admin Manages Products

1. **Precondition:** Admin is logged in.
2. **Main Flow:**
 - Admin navigates to "Products" dashboard.
 - Admin creates/edits/deletes products.
 - Admin updates stock levels.
3. **Alternative Flows:**
 - Admin filters products by category/price.
 - Admin exports product data to CSV.
4. **Postcondition:** Product inventory is updated.

Use Case 3: Staff Processes an Order

1. **Precondition:** Staff is logged in.
 2. **Main Flow:**
 - Staff views "Orders" dashboard.
 - Staff updates order status (e.g., "shipped").
 - Staff generates an invoice if not already done.
 3. **Alternative Flows:**
 - Staff cancels an order due to stock issues.
 - Staff resolves a customer dispute.
 4. **Postcondition:** Order status is updated; customer notified.
-

6. Success Criteria

6.1 Key Performance Indicators (KPIs)

Metric	Target
API Response Time	< 2s for 95% of requests.
System Uptime	99.9% availability.
User Registration	90% of users complete registration without errors.
Order Completion Rate	85% of carts are converted to orders.
Email Delivery Rate	95% of emails are successfully delivered.

6.2 Acceptance Criteria

Feature	Acceptance Criteria
User Registration	Users can register with valid email and password. Passwords are hashed before storage.
Product Management	Admins can CRUD products with categories. Product images are stored and displayed.
Order Processing	Users can add items to cart and checkout. Inventory is deducted on order completion.
Invoicing	Invoices are auto-generated for completed orders. Admins can mark invoices as paid/unpaid.
Email Notifications	Emails are sent for order confirmations and password resets.
Role-Based Access	Admins can access all features; staff/customers have restricted permissions.
Geographic Management	Admins can register states and cities for shipping.

6.3 Business Value Metrics

Metric	Business Impact
Revenue Growth	Increased sales through seamless checkout and product management.
Customer Satisfaction	Higher retention via order tracking and support features.
Operational Efficiency	Reduced manual work for order processing and inventory management.
Scalability	Ability to handle growth without performance degradation.
Cost Savings	Automated invoicing and email notifications reduce administrative overhead.

7. Implementation Timeline

7.1 High-Level Milestones

Phase	Duration	Deliverables
Discovery & Planning	2 weeks	BRD, user personas, technical architecture.
Backend Development	6 weeks	API endpoints, models, authentication, and core business logic.
Frontend Development	6 weeks	Next.js UI, cart, checkout, and admin dashboards.
Integration	3 weeks	Connect frontend to backend; test email and payment flows.
Testing	2 weeks	Unit, integration, and UAT testing.
Deployment	1 week	Deploy to production; monitor performance.
Go-Live	Ongoing	Post-launch support and iterative improvements.

7.2 Dependencies

- MongoDB Atlas Account (for database hosting).
- SMTP Credentials (for email sending).

- **Clouduinary Account** (for image storage).
- **Frontend Team** (for UI development).

7.3 Risk Considerations

Risk	Mitigation Strategy
Database Performance Issues	Use indexing for frequently queried fields (e.g., `userId` in `orders`).
Security Vulnerabilities	Regular security audits; use HTTPS and rate limiting.
Third-Party API Failures	Implement retry logic for email and payment services.
User Adoption	Conduct training sessions for admins/staff.
Scalability Bottlenecks	Monitor API response times; optimize queries with pagination.

8. Appendix

8.1 Glossary

Term	Definition
JWT	JSON Web Token: A secure way to authenticate users via tokens.
Soft Delete	Marking a record as deleted (e.g., <code>deletedAt: Date</code>) instead of physically removing it.
CRUD	Create, Read, Update, Delete: Basic operations for managing data.
RESTful API	A standard for designing networked applications using HTTP requests.
MongoDB Atlas	A cloud database service for MongoDB.

8.2 References

- Technical Docs:
 - [Express.js Documentation](#)
 - [MongoDB Atlas Guide](#)
 - [Next.js API Routes](#)
- Standards:
 - GDPR Compliance: [General Data Protection Regulation](#)
 - Security Best Practices: [OWASP Top 10](#)

Approval: | Role | Name | Signature | Date | |-----|-----|-----|-----| | Business
Owner | [Name] | | [Date] | | Tech Lead | [Name] | | [Date] |